

Structure

Structure is a user-defined data type in C which allows you to combine different data types to store a particular type of record. Structure helps to construct a complex data type in more meaningful way. It is somewhat similar to an Array. The only difference is that array is used to store collection of similar datatypes while structure can store collection of any type of data.

Defining a structure

struct keyword is used to define a structure. **struct** define a new data type which is a collection of different type of data.

Syntax :

```
struct structure_name
{
    //Statements
};
```

Example of Structure

```
struct Book
{
    char name[15];
    int price;
    int pages;
};
```

Here the **struct Book** declares a structure to hold the details of book which consists of three data fields, namely *name*, *price* and *pages*. These fields are called **structure elements or members**. Each member can have different data type, like in this case, **name** is of **char** type and **price** is of **int** type etc. **Book** is the name of the structure and is called structure tag.

Declaring Structure Variables

It is possible to declare variables of a **structure**, after the structure is defined. **Structure** variable declaration is similar to the declaration of variables of any other data types. Structure variables can be declared in following two ways.

1) Declaring Structure variables separately

```
struct Student
{
    char[20] name;
    int age;
    int rollno;
} ;

struct Student S1 , S2;    //declaring variables of Student
```

2) Declaring Structure Variables with Structure definition

```
struct Student
{
    char[20] name;
    int age;
    int rollno;
} S1, S2 ;
```

Here **S1** and **S2** are variables of structure **Student**. However this approach is not much recommended.

Accessing Structure Members

Structure members can be accessed and assigned values in number of ways. Structure member has no meaning independently. In order to assign a value to a structure member, the member name must be linked with the **structure** variable using dot **.** operator also called **period** or **member access** operator.

```
struct Book
{
    char name[15];
    int price;
```

```
int pages;  
} b1 , b2 ;
```

```
b1.price=200;           //b1 is variable of Book type and price is member of Book
```

We can also use `scanf()` to give values to structure members through terminal.

```
scanf(" %s ", b1.name);  
scanf(" %d ", &b1.price);
```

Structure Initialization

Like any other data type, structure variable can also be initialized at compile time.

```
struct Patient  
{  
    float height;  
    int weight;  
    int age;  
};
```

```
struct Patient p1 = { 180.75 , 73, 23 };    //initialization
```

or,

```
struct patient p1;  
p1.height = 180.75;    //initialization of each member separately  
p1.weight = 73;  
p1.age = 23;
```

Array of Structure

We can also declare an array of **structure**. Each element of the array representing a **structure** variable. **Example** : `struct employee emp[5];`

The above code define an array **emp** of size 5 elements. Each element of array **emp** is of type **employee**

```
#include<stdio.h>
#include<conio.h>
struct employee
{
    char ename[10];
    int sal;
};

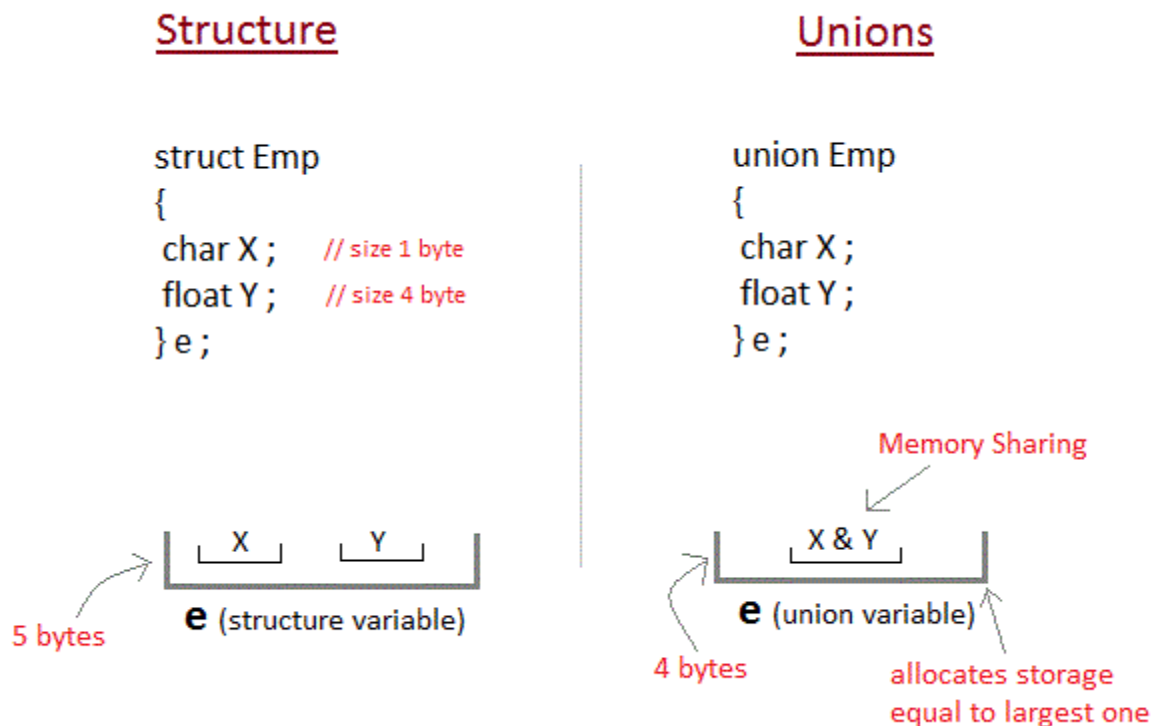
struct employee emp[5];
int i,j;
void ask()
{
    for(i=0;i<3;i++)
    {
        printf("\nEnter %dst employee record\n",i+1);
        printf("\nEnter employee name\t");
        scanf("%s",emp[i].ename);
        printf("\nEnter employee salary\t");
        scanf("%d",&emp[i].sal);
    }
    printf("\nDisplaying Employee record\n");
    for(i=0;i<3;i++)
    {
        printf("\nEnter employee name is %s",emp[i].ename);
        printf("\nEnter salary is %d",emp[i].sal);
    }
}

void main()
{
    clrscr();
```

```
ask();  
getch();  
}
```

Unions in C Language

Unions are conceptually similar to **structures**. The syntax of **union** is also similar to that of structure. The only difference is in terms of storage. In **structure** each member has its own storage location, whereas all members of **union** use a single shared memory location which is equal to the size of its largest data member.



This implies that although a **union** may contain many members of different types, **it cannot handle all the members at same time**. A **union** is declared using **union** keyword.

```
union item
{
    int m;
    float x;
    char c;
}It1;
```

This declares a variable **It1** of type union **item**. This **union** contains three members each with a different data type. However only one of them can be used at a time. This is due to the fact that only one location is allocated for a **union** variable, irrespective of its size. The compiler allocates the storage that is large enough to hold largest variable type in the **union**. In

the **union** declared above the member **x** requires 4 bytes which is largest among the members in 16-bit machine. Other members of **union** will share the same address.

Accessing a Union Member

Syntax for accessing **union** member is similar to accessing structure member,

```
union test
{
    int a;
    float b;
    char c;
}t;

t.a ;    //to access members of union t
t.b ;
t.c ;
```

Complete Example for Union

```
#include <stdio.h>
#include <conio.h>

union item
{
    int a;
    float b;
    char ch;
};

int main( )
{
```

```
union item it;

it.a = 12;

it.b = 20.2;

it.ch='z';

clrscr();

printf("%d\n",it.a);

printf("%f\n",it.b);

printf("%c\n",it.ch);

getch();

return 0;

}
```

Output :

-26426

20.1999

z

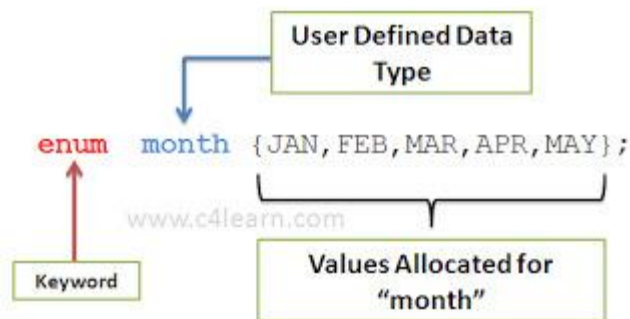
As you can see here, the values of **a** and **b** get corrupted and only variable **c** prints the expected result. Because in **union**, the only member whose value is currently stored will have the memory.

Enum [Enumerated] : User defined Data Type in C

Syntax:

```
enum identifier {value1, value2,... Value n};
```

- enum is **” Enumerated Data Type “**.
- enum is **user defined** data type
- In the above example **“identifier”** is nothing but the **user defined data type** .
- Value1, Value2, Value3..... etc creates **one set of enum values**.
- Using **“identifier”** we are **creating our variables**.



Let's Take Look at Following Example ...

Example :

```
enum month { JAN, FEB, MAR, APR, MAY, JUN, JUL, AUG, SEP, OCT, DEC };  
enum month rmonth;
```

- First Line Creates **“User Defined Data Type”** called **month**.
- It has 12 values as given in the **pair of braces**.
- In the second line “rmonth” variable is declared of type “month” which can be initialized with any “data value amongst 12 values”.

```
rmonth = FEB;
```

- Default Numeric value** assigned to first enum value is “0”.
- Numerically JAN is given value “0”.

3. FEB is given value “1”.
4. MAR is given value “2”.
5. APR is given value “3”.
6. MAY is given value “4”.
7. JUN is given value “5”.
8. and so on.....

```
printf("%d",rmonth);
```

- It will Print “1” on the screen because “Numerical Equivalent” of “FEB” is 1 and “rmonth” is initialized by “FEB”.
- Generally Printing Value of enum variable is as good as printing “**Integer**”.

Sample Program :

```
#include< stdio.h>
void main()
{
    int i;
    enum month {JAN,FEB,MAR,APR,MAY,JUN,JUL,AUG,SEP,OCT,DEC};
    clrscr();
    for(i=JAN;i<=DEC;i++)
        printf("\n%d",i);
}
```

Output :

```
01234567891011
```

Same notes presented in different manner for REFERENCE

Enumeration in C

An enumeration is a user-defined data type consists of integral constants and each integral constant is give a name. Keyword `enum` is used to defined enumerated data type.

```
enum type_name{ value1, value2,...,valueN };
```

Here, *type_name* is the name of enumerated data type or tag.
And *value1, value2,....,valueN* are values of type *type_name*.

By default, *value1* will be equal to 0, *value2* will be 1 and so on but, the programmer can change the default value.

```
// Changing the default value of enum elements

enum suit{

    club=0;

    diamonds=10;

    hearts=20;

    spades=3;

};
```

Declaration of enumerated variable

Above code defines the type of the data but, no any variable is created. Variable of type `enum` can be created as:

```
enum boolean{

    false;

    true;

};

enum boolean check;
```

Here, a variable `check` is declared which is of type `enum boolean`.

Example of enumerated type

```
#include <stdio.h>
enum week{ sunday, monday, tuesday, wednesday, thursday, friday, saturday};
int main(){
    enum week today;
    today=wednesday;
    printf("%d day",today+1);
    return 0;
}
```

Output

```
4 day
```

You can write any program in C language without the help of enumerations but, enumerations helps in writing clear codes and simplify programming.